IBM BOB 2.0 HACKATHON · SEPTEMBER 2026

BobFlow

Proof, not prompts.

An evidence-driven delivery workflow for IBM Bob 2.0

github.com/jadeji00/ibm-bobflow

THE PROBLEM

Generation became cheap. Trust did not.

- AI coding assistants collapsed development into a single gesture: describe what you want, receive code, and hope it works.
- The bottleneck never disappeared — it moved. Writing code is fast now. Knowing whether it is correct is not.
- AI-generated code arrives without the reasoning that produced it, so reviewing it becomes archaeology.
- The cost lands later — in code review, in incident response, in production.

THE INSIGHT

The problem is not that AI writes bad code.

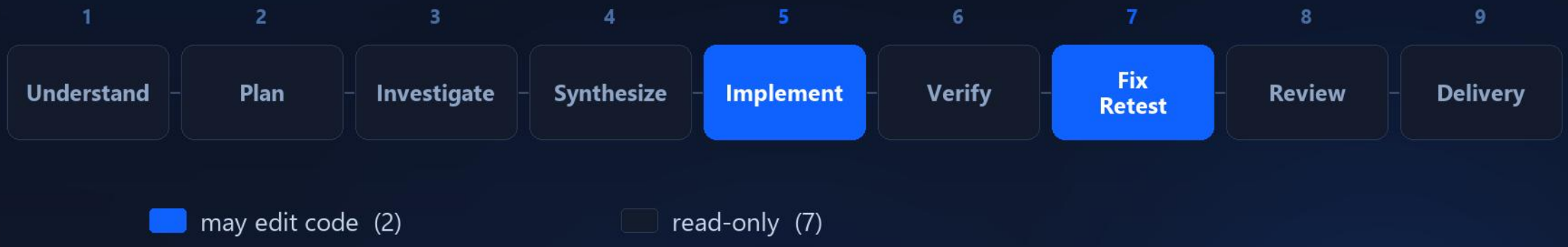
It is that AI writes code without evidence.

So BobFlow makes evidence a required deliverable — not an afterthought bolted on at the end.

THE WORKFLOW

BobFlow: nine explicit phases

Instead of one prompt producing one patch, work moves through nine phases. Each leaves an artifact — the chain of artifacts is the evidence.



THE CONSTRAINT

Only two phases may touch code

7 read-only

2 can write

- Everything before phase 5 is investigation.
- Everything after phase 5 is verification.
- No agent may verify its own work — the Security and Testing investigators never write the implementation they judge.

PHASE 3 — INVESTIGATION

Three independent investigations

Each specialist has a distinct mandate and its own output contract. None of them may edit a file.

Architecture

- Structure and entry points
- Data flow across layers
- Dependencies and impact
- Existing conventions

Security

- Attack surface
- Threat model
- Required controls
- Failure modes

Testing

- Test strategy
- Coverage mapping
- Threat → test traceability
- Edge cases

Their findings are reconciled in a synthesis step that resolves conflicts explicitly — never averages them.

THE TARGET CODEBASE

A real feature, delivered end-to-end

The demo app is not the point — it is the sample the workflow is applied to. Bob is the engine.

- A small full-stack application that works, but cannot upload avatars.
- One feature request: validated avatar upload.
- Stack: React + TypeScript + Vite · Express + TypeScript · Vitest + Supertest.
- The feature was specified, investigated, implemented and verified entirely by IBM Bob.

WHAT SECURITY FOUND

The threat model drove the tests

The Security investigator enumerated the upload attack surface. Testing mapped every threat to a concrete case.

01	**Size-limit bypass**	Oversized files exhausting storage and memory.
02	**MIME spoofing**	A client declaring a type it does not actually send.
03	**Magic-byte mismatch**	A text file renamed .png to pass the extension check.
04	**Path traversal**	Crafted filenames escaping the upload directory.
05	**Storage-name control**	Client-supplied names overwriting existing files.

VERIFICATION

Verification means execution

Inferring a test result from reading code is not verification. The raw terminal output is the deliverable.

28 / 28

tests passing · 2 test files · 0 failures

- Bob executed npm test and showed the output — it was not summarised afterwards.
- Bob then explicitly declined phase 7, Fix/Retest: the suite was green on the first run.
- It did not invent a failure narrative to look more impressive. That restraint is the workflow working.

Bob at every stage

/init	Four project context files: AGENTS.md plus per-mode rules for agent, ask and plan.
Plan mode	Phases 1–2 — understanding and planning, zero edits.
Read-only tasks	Phase 3 — Architecture, Security and Testing investigations.
Agent mode	Phases 5 and 7 — the only phases permitted to modify code.
Execution	Phase 6 — npm test, run for real rather than predicted.

Most agent frameworks optimise for autonomy.

BobFlow optimises for auditability.

- The cost is deliberate: read-only investigation is slower than letting an agent start writing immediately.
- In exchange, the delivery artifact is something a reviewer can actually check.
- Fewer decisions made by the machine, more decisions that can be audited afterwards.

BobFlow

Proof, not prompts.

github.com/jadeji00/ibm-bobflow

IBM Bob 2.0 Hackathon · September 2026